# Supermarket Database — Design Specification

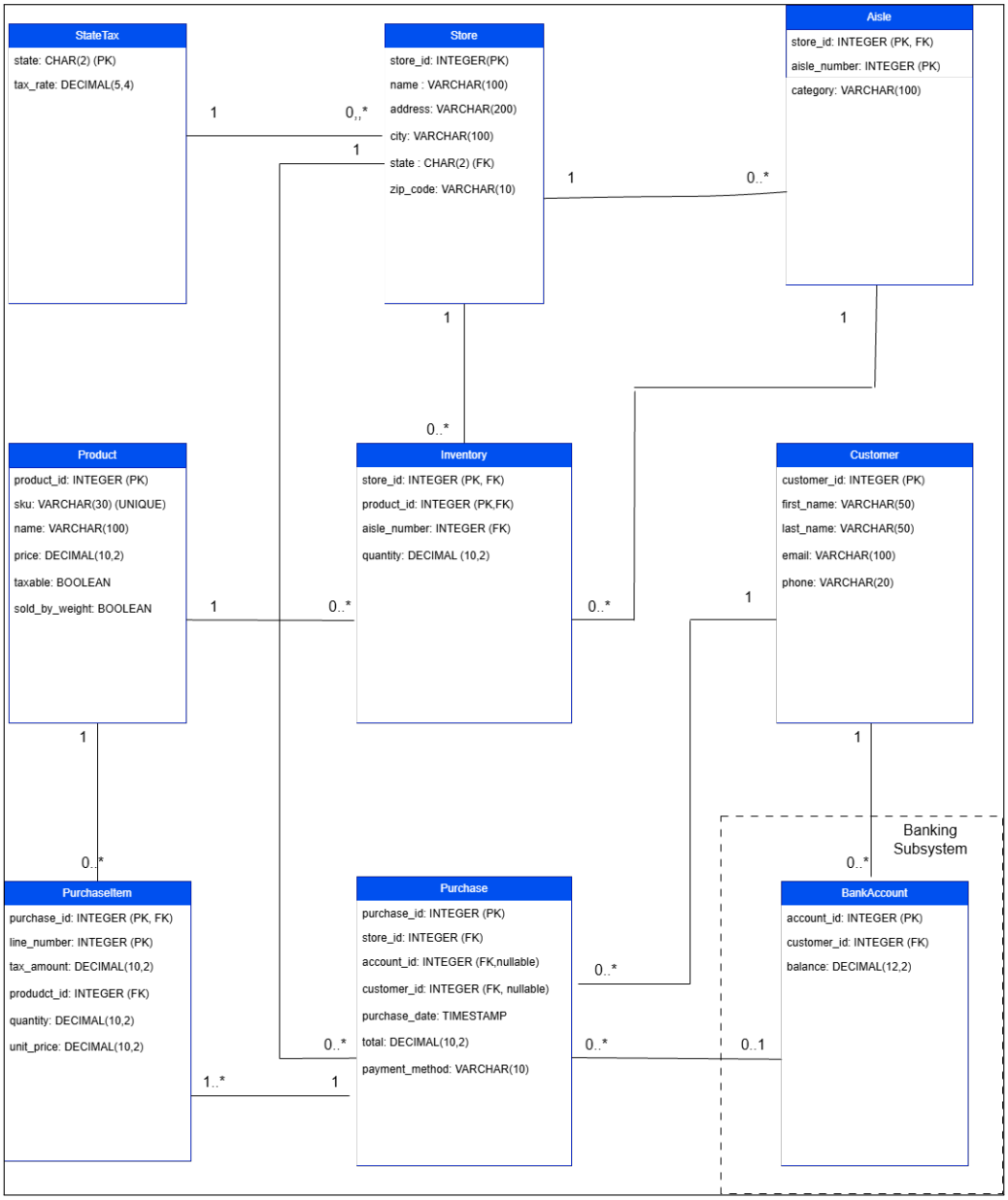



Figure 1: Supermarket Database ER Model Draft

## 1 Project Overview

This project implements a relational database for a supermarket chain using PostgreSQL.

The system will simulate supermarket checkout operations, including product purchases, inven-

tory management, sales tax calculations, receipt generation, and payment processing.

The database will support a JavaScript web application with customer-facing checkout operations and back-office reporting.

## 2 Database Requirements

The database must support:

- Multiple supermarket locations across at least four states.
- At least 100 stores, each containing 20–40 aisles.
- Products identified by SKU.
- Store-specific inventory quantities.
- Products sold by unit or weight.
- State-dependent sales taxes, including tax-exempt products.
- Cash and card payments.
- Purchase receipts containing individual items and totals.
- Card payments that update a separate bank account table.
- SQL transactions that modify multiple tables atomically.
- Reporting queries involving joins and aggregations.

## 3 Preliminary Database Entities

| Entity | Description |
|---|---|
| Store | Physical supermarket location |
| Product | Product catalog containing SKU, price, and tax status |
| Inventory | Stock quantities for products at individual stores |
| Customer | Customer identification and associated information |
| BankAccount | Account balances used for simulated card payments |
| Purchase | Checkout transaction and receipt summary |
| PurchaseItem | Individual receipt line items |
| StateTax | State-level sales tax rates |

Table 1: Core Preliminary Database Entities

Additional entities, such as `Aisle`, may be introduced during ER refinement.

## 4 Preliminary Relationships

- One Store has many Inventory records.
- One Product can appear in many Inventory records.
- One Store processes many Purchases.

- One Customer can make many Purchases.
- One Purchase contains many PurchaseItems.
- One Product can appear in many PurchaseItems.
- One Customer may have an associated BankAccount.
- One StateTax record may apply to multiple Stores.

## 5 Primary and Foreign Keys

The schema will use surrogate primary keys for major business entities.

Inventory will use a composite primary key:

`(store_id, product_id)`

PurchaseItem will use a composite primary key:

`(purchase_id, line_number)`

Foreign keys will enforce referential integrity across all related tables.

## 6 Transaction Processing

The primary database transaction simulates supermarket checkout. Operations include:

1. Validate the selected products and quantities.
2. Verify sufficient inventory.
3. Calculate the purchase total, including applicable taxes.
4. Verify sufficient funds for card payments.
5. Insert the purchase record.
6. Insert individual purchase items.
7. Reduce inventory quantities.
8. Deduct the payment amount from the bank account when paying by card.
9. Commit all changes together.

If any operation fails, the transaction must roll back.

## 7 Normalization

The database will target Third Normal Form (3NF), with Boyce-Codd Normal Form (BCNF) considered where appropriate.

Product information, store information, inventory, customer information, and purchase records will be stored separately to reduce redundancy and update anomalies.

Receipt line items will preserve historical sale prices, even if current product prices change.

# 8 Planned SQL Reports

The initial application will support at least three JOIN-based reporting queries:

1. Sales totals by store.
2. Product sales quantities and revenue.
3. Customer purchase history with receipt totals.

# 9 Development Status

## 9.1 Phase 1: Preliminary Implementation

**Status:** Design in progress

**Rubric Requirements:** Should have a preliminary ER model covering the minimum attributes listed above, at least five SQL tables. Payment with card must be recorded on a bank account table, subtracting the amount from the balance. You should develop a basic web app functionality sending SQL to the DBMS: at least one transaction code block and at least 3 insightful queries, web app working locally. The transaction must update at least two tables (one of them should be the bank account). All queries should involve joins, which assumes your database ER model is normalized.

The ER model should include the most detailed table for each target application: (1) detail cellphone calls (duration), data usage (MBs) and total minutes/data and amount to pay. (2) Each item in the receipt (price, quantity or weight) and total amount to pay. Phase 1 will be PASS/FAIL. A team receiving FAIL cannot submit Phase 2.

The preliminary schema and relationships are subject to revision during ER modeling and normalization.

| Status | Task Requirement | Completed By |
|---|---|---|
| **Delegation and Architecture** | | |
| ■ | UML ER Draft | Mark |
| ☐ | Audit & finalize preliminary model | TBD |
| ☐ | Lock in the schema (exact table names, column names, data types) | TBD |
| **Database and SQL Build** | | |
| ☐ | Initialize database locally | TBD |
| ☐ | Write DDL CREATE TABLE scripts with primary/foreign keys | TBD |
| ☐ | Write INSERT scripts for dummy data | TBD |
| ☐ | Write transaction block updating two tables concurrently | TBD |
| ☐ | Write reporting queries using JOIN operations | TBD |
| **Backend Logic** | | |
| ☐ | Configure local database connection | TBD |
| ☐ | Create routes for frontend to call | TBD |
| ☐ | Test endpoints to verify DB access | TBD |
| **Frontend GUI Build** | | |
| ☐ | Build basic HTML skeleton with required forms and buttons | TBD |
| ☐ | Write calls to backend | TBD |
| ☐ | Implement buttons to drop/create/insert | TBD |
| ☐ | Implement view function | TBD |
| ☐ | Implement simulation button for concurrent transaction | TBD |
| **Phase 1 Validation** | | |
| ☐ | End-to-end test | TBD |
| ☐ | Force OLTP error to verify error handling | TBD |
| ☐ | Verify Phase 1 rubric requirements | TBD |

### 9.2 Phase 2: Completed Web-App

**Status:** Not Started

**Rubric Requirements:** a fully working web app, with a polished DB model, well-designed ER model complying with ER-to-relational mapping, with all tables updated with transactions (no isolated INSERT or UPDATE). The project report PDF will be submitted 1 day after the web app is submitted (to give time for text polishing, but no changes allowed in source code).

| Status | Task Requirement | Completed By |
|---|---|---|
| **Database & Schema** | | |
| ☐ | Normalize to 3NF/BCNF | TBD |
| ☐ | Build tables for store, state, aisle, product, and tax requirements | TBD |
| ☐ | Lookup tables with minimum 10 rows | TBD |
| **Backend** | | |
| ☐ | Back office reporting for money totals | TBD |
| ☐ | .sql files | TBD |
| **Frontend** | | |
| ☐ | Automate random selection, compute, and process | TBD |
| ☐ | OLTP error and dangerous display messages | TBD |

### 9.3 Phase 3: Documentation

**Status:** Not Started

**Requirements:**

- **Concurrency Testing:** Ensure compatibilty across multiple browsers and concurrent requests
- **PDF:** call it "teamXY.pdf" where XY is your team id (01,02,..,21,..30). Report in PDF format including ER diagrams, written summary of web app including SQL for transaction and SQL for a few important queries (i.e. the most importan tables), and http link to demo video. Your report should include a list of web references at the end, including websites you chose as reference to build your app.
- **Source Files:** verify all source code, diagram files and any useful document you got from the Internet. Source code and web app originality: It is expected all source code, databases and GUI will be different even if some source code, some tables, and website ideas are obtained from the Internet
- **Video:** 8 minutes max - stored in some public website (youtube, dropbox, Google drive, etc). No password please, long URL OK, make sure it opens by anyone in one click. Format: mp4, viewable on any browser. Do not upload video files to our servers as it takes too much space and this violates UH IT policies. You should record a video, covering both DB model, important SQL (transaction code block and a few queries) and showing the finished web application in Phase 2. We will have a Teams session where all teams, TAs and the professor will watch it. Format: MP4.